

CLASIFICACIÓN DE RESIDUOS

Juan Gómez Díaz

1. Introducción y Definición del Problema

Este proyecto aborda el problema de clasificación de imágenes de residuos mediante técnicas de aprendizaje automático y profundo. El objetivo es clasificar automáticamente imágenes de basura en seis categorías predefinidas, una tarea de gran relevancia para la automatización del reciclaje y los sistemas inteligentes de gestión de residuos.

2. Dataset

Se utilizo el dataset Garbage Classification disponible en Kaggle con un total de aproximadamente 2.500 fotos de 6 clases diferentes de basura(Cardboard(Cartón), Glass(Vidrio), Metal, Plastic(Plástico) y Trash(Basura en general).

3. Métricas de Evaluación

Se utilizaron las siguientes métricas de evaluación, reportadas por clase.

3.1 Métricas Principales

Métrica	Fórmula	Justificación
Accuracy	TP_total / N_total	Vista global del rendimiento en todas las clases.
Precision	$TP / (TP + FP)$	Fiabilidad de las predicciones positivas por clase.
Recall	$TP / (TP + FN)$	Proporción de muestras reales de cada clase detectadas correctamente.
F1-score	$2 * (P * R) / (P + R)$	Balance entre Precision y Recall; útil con clases no perfectamente balanceadas.

4. Modelos

Al principio del proyecto se realizaron modelos baseline simples y después se pasó a hacer arquitecturas más complejas de deep learning. Se han acabado desarrollando un total de doce modelos.

4.1 Modelo Lineal Baseline — Regresión Logística

El primer modelo sirve como baseline lineal para establecer un punto de partida mínimo. Las imágenes son aplanadas a un vector único antes de aplicar el clasificador.

- Modelo: LogisticRegression de scikit-learn
- Preprocesamiento: Redimensionado a 64×64 píxeles + aplanado a vector de 12.288 valores
- Número de parámetros: 294.918

Resultados

Train Acc.	Val. Acc.	Test Acc.
1.0000	0.4000	0.4500

El modelo muestra un claro sobreajuste: accuracy perfecto en entrenamiento pero solo 45% en test. Esto confirma que las características lineales obtenidas a partir del vector de píxeles aplanados son insuficientes para capturar la complejidad visual de las imágenes de residuos.

Reporte de Clasificación — Test

Clase	Precision	Recall	F1-score
cardboard	0.63	0.66	0.64
glass	0.33	0.32	0.33
metal	0.17	0.10	0.12
paper	0.64	0.64	0.64
plastic	0.38	0.51	0.43
trash	0.23	0.23	0.23
accuracy			0.45
macro avg	0.40	0.41	0.40
weighted avg	0.43	0.45	0.43

4.2 Modelo Machine Learning — Random Forest

El segundo modelo aplica un Random Forest con búsqueda de hiperparámetros mediante GridSearchCV (validación cruzada de 3 particiones) sobre las mismas features de píxeles aplanados.

- Modelo: RandomForestClassifier con GridSearchCV
- Grid buscado: $n_estimators \in \{50, 100, 150\}$, $max_depth \in \{10, 15, 20, 25\}$, $min_samples_split \in \{2, 5, 10\}$, $min_samples_leaf \in \{1, 2, 4\}$
- Número de parámetros: ~65.056

Resultados

Train Acc.	Val. Acc.	Test Acc.
0.9980	0.6324	0.6640

El Random Forest mejora notablemente respecto a la regresión logística, aunque sigue sobre ajustando. Sigue siendo insuficiente para una aplicación real de clasificación de residuos.

Reporte de Clasificación — Test

Clase	Precision	Recall	F1-score
cardboard	0.68	0.73	0.71
glass	0.55	0.58	0.56
metal	0.60	0.44	0.51
paper	0.71	0.86	0.78
plastic	0.74	0.71	0.73
trash	0.71	0.38	0.50
accuracy			0.66
macro avg	0.67	0.62	0.63
weighted avg	0.66	0.66	0.66

4.3 Redes Neuronales Convolucionales desde Cero

Se comenzó con el diseño de una red neuronal convolucional (CNN) básica, y se fue incrementando la profundidad y complejidad de la arquitectura de forma progresiva. Con el objetivo de que el modelo fuera capaz de aprender características más abstractas y complejas, para distinguir las sutiles diferencias visuales entre los distintos tipos de residuos.

CNN Básica ("Model 0")

La red más básica posible, con un único bloque convolucional y un clasificador directo. Su propósito es confirmar que una CNN mínima ya aprende representaciones espaciales básicas.

- Arquitectura: 1 capa Conv2D (tantos filtros como clases) → GlobalAveragePooling2D → Dense(6, softmax)
- Parámetros: 210
- Resultados: Train 0,37 | Val 0,38 | Test 0,38

CNN Model 1

Ligero incremento de capacidad respecto al modelo básico, añadiendo un bloque convolucional con 32 filtros.

- Arquitectura: Conv2D(32) → GlobalAveragePooling2D → Dense(6, softmax)
- Parámetros: 1.094
- Resultados: Train 0,6165 | Val 0,6087 | Test 0,6443
- Mejora significativa respecto al modelo básico. El modelo empieza a poder extraer características espaciales representativas para la clasificación

CNN Model 2

Incremento sustancial de capacidad añadiendo múltiples bloques convolucionales con mayor número de filtros, Dropout y capas densas intermedias.

- Arquitectura: Conv2D(32) → Conv2D(64) → Conv2D(128) → GlobalAveragePooling2D → Dropout(0,2) → Dense(6)
- Parámetros: 94.022
- Resultados: Train 0,9193 | Val 0,8735 | Test 0,8300
- Gran salto en rendimiento. El modelo ya es capaz de hacer una extracción profunda de características.
- Aunque el modelo en Train empiece a llegar a 1, la validación se acerca no como nos pasaba en el modelo 1

CNN Model 3

Arquitectura más profunda con más filtros en las capas convolucionales y mayor capacidad.

- Arquitectura: Conv2D(32) → Conv2D(64) → Conv2D(128) → Conv2D(256) → GlobalAveragePooling2D → Dropout(0,2) → Dense(6)
- Parámetros: 389.958
- Resultados: Train 0,9980 | Val 0,8854 | Test 0,8538
- Buen rendimiento pero comienza el sobreajuste.

CNN Model 4

En este modelo, decidimos aumentar el ancho de la red desde la primera etapa. Al sustituir la capa inicial de 32 filtros por una configuración de dos capas consecutivas de 64 filtros, buscamos incrementar la capacidad de extracción de características de bajo nivel (bordes, texturas y colores).

- Arquitectura: Conv2D(64) → Conv2D(64) → Conv2D(128) → Conv2D(256) → GlobalAveragePooling2D → Dropout(0,1) → Dense(6)
- Parámetros: 409.286
- Resultados: Train 0,7996% | Val 0,7075 | Test 0,7708
- Peor resultado respecto al Model 3.

CNN Model 5

Se introdujo una capa densa intermedia con activación ReLU antes de la capa de salida. Con el objetivo de tener un extractor de características, permitiendo que el modelo combine las representaciones espaciales obtenidas por las capas convolucionales y aprenda relaciones no lineales complejas.

- Arquitectura: Conv2D(64) → Conv2D(64) → Conv2D(128) → Conv2D(256) → GlobalAveragePooling2D → Dense(256) → Dropout(0,3) → Dense(6)
- Parámetros: 474.822
- Resultados: Train 0,8976 | Val 0,8458 | Test 0,8103
- Mejora respecto al Model 4 pero inferior al Model 3

4.4 Transfer Learning — ResNet50

Se empleó Transfer Learning usando ResNet50 preentrenada en ImageNet. Se experimentaron dos estrategias: fine-tuning completo y base congelada.

CNN Model 6 — ResNet50 (Fine-tuning completo)

La base completa de ResNet50 se conectó a una cabeza de clasificación personalizada y se entrenó con fine-tuning de todos los pesos.

- Arquitectura: ResNet50 (base completa) → GlobalAveragePooling2D → Dense(256) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(6, softmax)
- Parámetros: 24.147.078
- Resultados: Train 0,9990 | Val 0,8933 | Test 0,8735

CNN Model 7 — ResNet50 (Base congelada)

La base de ResNet50 se mantuvo congelada y solo se entrenó la cabeza de clasificación.

- Arquitectura: → GlobalAveragePooling2D → Dense(256) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(6, softmax)
- Parámetros: 558,598
- Resultados: Train 0.9985 | Val 0,8854 | Test 0,8656
- El test accuracy del Model 7 es ligeramente inferior al Model 6, confirmando que el fine-tuning completo aporta algo de mejora

Reporte de Clasificación — Model 7 (ResNet50 frozen) — Test

Clase	Precision	Recall	F1-score
cardboard	0.95	0.95	0.95
glass	0.81	0.88	0.85
metal	0.82	0.90	0.86
paper	0.95	0.95	0.95
plastic	0.89	0.83	0.86
trash	0.89	0.57	0.70
accuracy			0.87
macro avg	0.89	0.85	0.86
weighted avg	0.89	0.89	0.88

La clase 'trash' sigue siendo la más difícil de clasificar con ambas variantes de ResNet50, con recall de 57-62%, lo que sugiere que esta clase es heterogénea y difícil.

4.5 Transfer Learning — EfficientNetB0 (Model 8)

EfficientNetB0 es una arquitectura moderna muy eficiente que combina escalado compuesto de profundidad, anchura y resolución. Con solo 4.4M de parámetros (vs 24M de ResNet50), logra un excelente rendimiento.

- Arquitectura: EfficientNetB0 (preentrenada ImageNet) → GlobalAveragePooling2D → Dense(256) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(6, softmax)
- Parámetros totales: 4.427.177
- Data augmentation aplicado durante el entrenamiento. Es decir a la imágenes que entran se les aplica una transformación pero no aumenta el numero de imágenes con las que entrenamos.

Resultados — Semilla 42

Train Acc.	Val. Acc.	Test Acc.
0.9990	0.9209	0.8972

Reporte de Clasificación — Model 8 — Test (Semilla 42)

Clase	Precision	Recall	F1-score	Support
cardboard	0.98	0.98	0.98	41
glass	0.88	0.92	0.90	50
metal	0.95	0.88	0.91	41
paper	0.90	0.90	0.90	59
plastic	0.86	0.90	0.88	49
trash	0.67	0.62	0.64	13
accuracy			0.90	253
macro avg	0.87	0.86	0.87	253
weighted avg	0.90	0.90	0.90	253

4.6 Transfer Learning — DenseNet121 (Model 9)

DenseNet121 introduce conexiones densas entre capas (cada capa recibe los feature maps de todas las capas anteriores), lo que mejora el flujo de gradientes y facilita el reuso de características. Es el modelo con mejor rendimiento del proyecto.

- Arquitectura: DenseNet121 (preentrenada ImageNet) → GlobalAveragePooling2D → Dense(256) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(128) → Dropout(0,4) → Dense(6, softmax) Parámetros totales: 7.349.574

Resultados — Semilla 42

Train Acc.	Val. Acc.	Test Acc.
0.9960	0.9289	0.9051

Reporte de Clasificación — Model 9 — Test (Semilla 42)

Clase	Precision	Recall	F1-score
cardboard	0.89	0.95	0.92
glass	0.98	0.86	0.91
metal	0.76	0.93	0.84
paper	0.93	0.95	0.94
plastic	1.00	0.88	0.93
trash	0.83	0.77	0.80
accuracy			0.91
macro avg	0.90	0.89	0.89
weighted avg	0.91	0.91	0.91

5. Comparativa Global de Modelos

La siguiente tabla resume el rendimiento de todos los modelos desarrollados, ordenados por complejidad creciente. Los modelos marcados con ✓ corresponden a los mejores modelos del proyecto.

Modelo	Parámetros	Train Acc.	Val. Acc.	Test Acc.
Regresión Logística	294.918	1.0000	0.4000	0.4500
Random Forest (GridSearchCV)	65.056*	0.9980	0.6324	0.6640
CNN Básica (Model 0)	210	0.3700	0.3800	0.3800
CNN Model 1	1.094	0.6165	0.6087	0.6443
CNN Model 2	94.022	0.9193	0.8735	0.8300
CNN Model 3	389.958	0.9980	0.8854	0.8538
CNN Model 4	409.286	0.7996	0.7075	0.7708
CNN Model 5	474.822	0.8976	0.8458	0.8103
CNN Model 6 (ResNet50)	24.147.078	0.9990	0.8933	0.8735
CNN Model 7 (ResNet50 frozen)	558,598	0.9985	0.8854	0.8656
CNN Model 8 (EfficientNetB0) ✓	4.427.177	0.9990	0.9209	0.8972
CNN Model 9 (DenseNet121) ✓	7.349.574	0.9960	0.9289	0.9051

Observaciones clave sobre la evolución del rendimiento:

- Los modelos lineales (Regresión Logística, Random Forest) presentan sobreajuste y bajo rendimiento en test, evidenciando que las features de píxeles aplanados no capturan la relaciones espaciales y eso hace que el modelo no generalice bien.
- Las CNNs desde cero muestran una mejora progresiva con mayor capacidad: el Model 2 con 94K parámetros ya alcanza 83% en test.
- Los modelos Model 4 y 5 no mejoran al Model 3, lo que sugiere que el aumento masivo de la arquitectura no da mejores resultados.
- Los modelos de Transfer Learning (Models 6-9) dan el mayor salto cualitativo, confirmando el valor del conocimiento preentrenado en ImageNet.
- EfficientNetB0 (Model 8) y DenseNet121 (Model 9) son los mejores modelos, siendo DenseNet121 el óptimo con 90.51% en test.

6. Evaluación de Robustez con Diferentes Semillas

Para los dos mejores modelos se realizó una evaluación de robustez repitiendo el experimento con 4 semillas diferentes (42, 123, 7, 99). Esto permite estimar la variabilidad del rendimiento debida a la aleatoriedad del proceso de entrenamiento.

6.1 Model 8 (EfficientNetB0) — Evaluación Multi-Semilla

Semilla	Train Acc.	Val. Acc.	Test Acc.
42	0.9990	0.9249	0.9012
123	0.9980	0.9012	0.8617
7	0.9985	0.9249	0.8775
99	0.9985	0.8893	0.8656
Media	0.9985	0.9101	0.8765

La variabilidad del Test Accuracy de EfficientNetB0 entre semillas es de ± 1.97 puntos porcentuales (rango: 86.17% - 90.12%). La semilla 42 ofrece el mejor resultado individual con 90.12%.

6.2 Model 9 (DenseNet121) — Evaluación Multi-Semilla

Semilla	Train Acc.	Val. Acc.	Test Acc.
42	0.9990	0.8972	0.9012
123	0.9990	0.8893	0.8854
7	0.9990	0.8893	0.8696
99	0.9990	0.8933	0.8972
Media	0.9990	0.8923	0.8883

DenseNet121 muestra un Train Accuracy constante de 99.90% en todas las semillas. La variabilidad del Test Accuracy es de ± 1.58 puntos porcentuales (rango: 86.96% - 90.12%), siendo más estable que EfficientNetB0.

DenseNet121 resulta ser el modelo más robusto y con mejor rendimiento medio, a pesar de tener mayor número de parámetros. Su media de Test Accuracy (88.83%) supera a EfficientNetB0 (87.65%).

7. Análisis final

7.1 Evolución del Rendimiento

El análisis de la evolución del rendimiento a lo largo de los modelos revela varias tendencias claras:

- El salto más grande se produce al pasar de modelos lineales/ML a CNNs desde cero: de 66% (Random Forest) a 83% (CNN Model 2). Esto confirma que las representaciones espaciales son fundamentales para este tipo de problema.
- El segundo salto importante ocurre con el Transfer Learning. EfficientNetB0 y DenseNet121 logran 89-91% en test, superando notablemente a las CNNs entrenadas desde cero.
- Los modelos de Transfer Learning con arquitecturas eficientes (EfficientNetB0) son competitivos con modelos más grandes (ResNet50) usando 5.4 veces menos parámetros.

7.2 Clases Problemáticas

El análisis de los reportes de clasificación revela un patrón consistente en todos los modelos:

- 'Trash' (basura general) es la clase más difícil, con recall entre 57-77% incluso en los mejores modelos. Esta clase agrupa objetos muy heterogéneos que no pertenecen a otras categorías, lo que la hace difícil de definir visualmente.
- 'Glass' y 'Metal' también presentan mayor confusión, posiblemente por similitud visual en texturas y colores.
- 'Cardboard' y 'Paper' son las clases mejor clasificadas en los modelos avanzados, con F1-score de 0.92-0.98.

7.3 Sobreajuste vs. Generalización

Un patrón recurrente en todos los modelos es el sobreajuste: el accuracy de entrenamiento es casi siempre superior al de validación y test. Las estrategias empleadas para mitigarlo fueron:

- Dropout en las capas densas de clasificación
- ReduceLROnPlateau: reducción automática del learning rate cuando la validación se estanca
- EarlyStopping con restauración de la mejor época (modelos 8 y 9)
- Data augmentation (modelos de Transfer Learning)

8. Mejoras

Para mejorar aún más el rendimiento:

- Data Augmentation más agresivo: recortes aleatorios, ajustes de color, flip vertical, rotaciones mayores.
- Aumento del dataset con imágenes adicionales de la clase 'trash'.
- Prueba de arquitecturas más modernas: EfficientNetV2, ConvNeXt, Vision Transformers (ViT).